

Compare Roboflow And YOLO

Today 11:52 AM



Roboflow vs YOLO

This is manual inspection as a part of property inspection

For your **property inspection use case**, Roboflow vs YOLO is not really an either/or comparison.

The simple answer

Roboflow = the computer-vision platform and dataset workflow

YOLO = the AI model that performs the detection

For the wall defect in your image—small scratches/scuff marks highlighted manually—you will likely get the best result by using **Roboflow for annotation/dataset management + YOLO for the actual model**, or by building the whole pipeline yourself with YOLO.

Roboflow supports dataset upload, annotation, versioning, augmentation, training, and deployment workflows, while its training platform can train multiple architectures, including YOLO models.

 [Roboflow Docs+1](#)

Your example: manual vs AI inspection

Current manual process

```
Inspector takes photo
↓
Inspector looks at wall
↓
Inspector finds scratch
↓
Inspector marks defect manually
↓
```

```
Inspector selects defect type
↓
Inspector submits inspection
```

For your image:

```
Wall Image
↓
Manual inspector sees scratches
↓
Marks the area with red annotation
↓
Creates defect:
Type = Scratch / Surface Damage
```

Proposed AI-assisted process

```
Inspector takes photo
↓
Upload to .NET API
↓
YOLO / Vision Model analyzes image
↓
AI detects potential defects
↓
AI returns coordinates + confidence
↓
React displays bounding box / mask
↓
Inspector confirms or rejects
↓
Final defect saved in SQL Server
```

For example:

```
JSON
{
  "defects": [
    {
      "type": "Wall Scratch",
      "confidence": 0.87,
      "boundingBox": {
        "x": 190,
        "y": 180,
        "width": 150,
        "height": 110
      },
      "status": "AI_Suggested"
    }
  ]
}
```

The important design decision for your property handover system is:

AI should suggest the defect; the inspector should approve it.

That is much safer than fully automatic inspection, especially when defects can affect handover decisions and contractor liability.

Roboflow vs YOLO for your project

Area	Roboflow	YOLO / Ultralytics
What it is	Computer vision platform	Model/framework
Image annotation	Excellent	You manage separately
Dataset versioning	Built-in	Manual
Augmentation	Built-in	Code/config
Training	Easy cloud/UI workflow	Python/CLI/cloud
Custom defect detection	Yes	Yes
Deployment	Managed or self-hosted	Self-hosted/exported
.NET integration	REST/Inference options	ONNX Runtime is a strong option
Data control	Depends on deployment choice	Maximum control
Vendor dependency	Higher	Lower
Best for	Fast experimentation	Production control

Roboflow provides managed and self-hosted deployment options and can run workflows on your own infrastructure, while Ultralytics supports training, validation, prediction, segmentation, and export formats such as ONNX. [Roboflow Docs+2](#)

My recommendation for your Property Inspection System

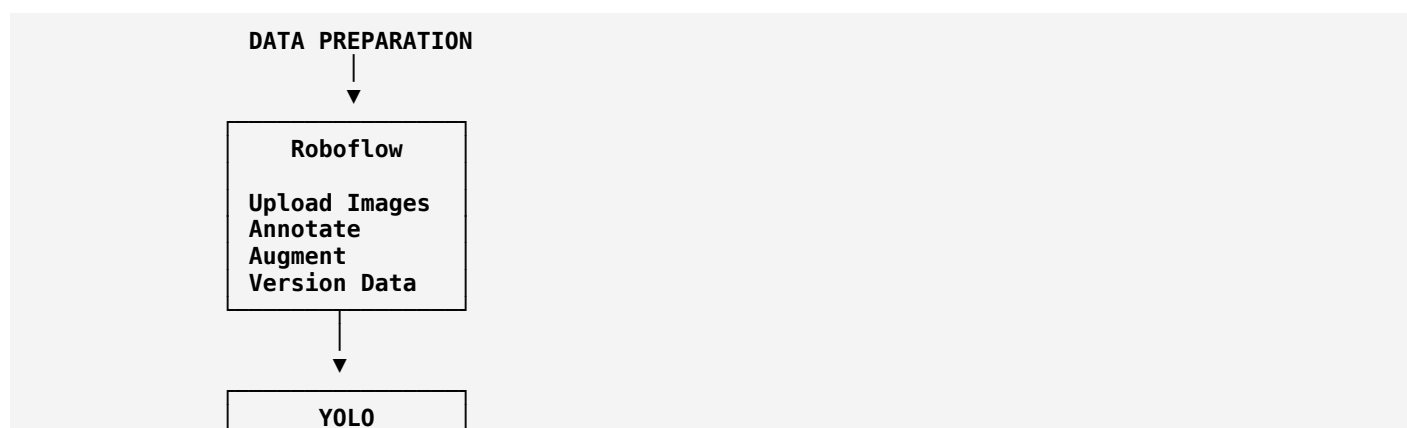
Given your existing architecture:

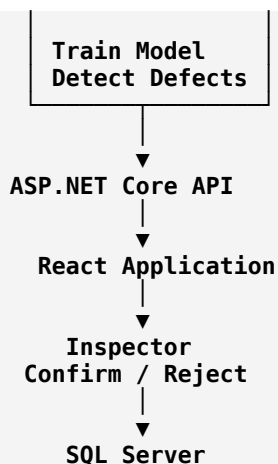
- **ASP.NET Core / .NET**
- **React frontend**
- **SQL Server**
- Existing property handover and inspection workflows
- Need for custom domain-specific defects

I recommend:

Option 1 — Best starting approach

Roboflow + YOLO





This is the fastest way to build a **proof of concept**.

You can annotate the wall defects visually in Roboflow, create dataset versions, train/test models, and then evaluate whether YOLO can reliably detect your actual defects. Roboflow also supports deploying models/workflows in cloud or on your own hardware. [Roboflow Docs+1](#)

The biggest challenge in your image

The scratches in your example are **small and subtle**.

This is important because generic object detection is easier for things like:

- Door
- Window
- AC unit
- Fire extinguisher
- Missing tile

But your use case includes:

- Small scratches
- Hairline cracks
- Paint defects
- Dampness
- Stains
- Chips
- Scuff marks

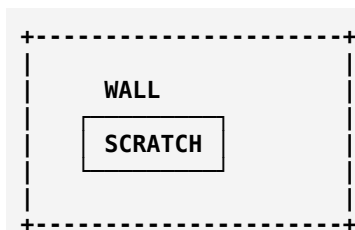
These can occupy only a very small percentage of the image.

Standard object detection may not be enough

For this image, I would test **two approaches**.

Approach A: YOLO Object Detection

Annotate defects using boxes:

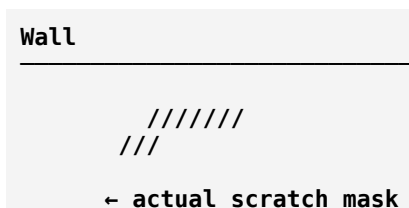


Good for:

- Holes
- Large cracks
- Missing components
- Broken switches
- Visible stains

Approach B: YOLO Segmentation

Annotate the **actual defect pixels/shape** rather than a rectangular box.



For irregular defects such as scratches and cracks, segmentation can provide a more precise representation than a bounding box. Ultralytics supports instance segmentation training and exporting segmentation models for deployment. [Ultralytics Docs](#)

For your specific image, I would investigate segmentation.

The defect is thin, irregular, and relatively small. A large bounding box would contain mostly normal wall area, which can make labeling and training less precise.

Recommended defect classes

I would not initially create 30–40 defect types. Start with a manageable dataset:



Floor

- Crack
- Scratch
- Broken Tile
- Stain

Door

- Scratch
- Dent
- Broken
- Misalignment

Ceiling

- Crack
- Water Damage
- Paint Damage

Electrical

- Broken Switch
- Missing Cover
- Damage

Start with **5–10 high-frequency defect classes**.

Dataset is more important than choosing Roboflow or YOLO

For your image specifically, the model needs examples such as:

Scratch on white wall
 Scratch on beige wall
 Scratch under bright light
 Scratch under low light
 Small scratch
 Large scratch
 Single scratch
 Multiple scratches
 Scratch near switch
 Scratch near door

You also need **negative images**:

Clean white wall
 Clean wall with shadows
 Wall texture
 Wall reflection
 Natural paint variations
 Lighting artifacts

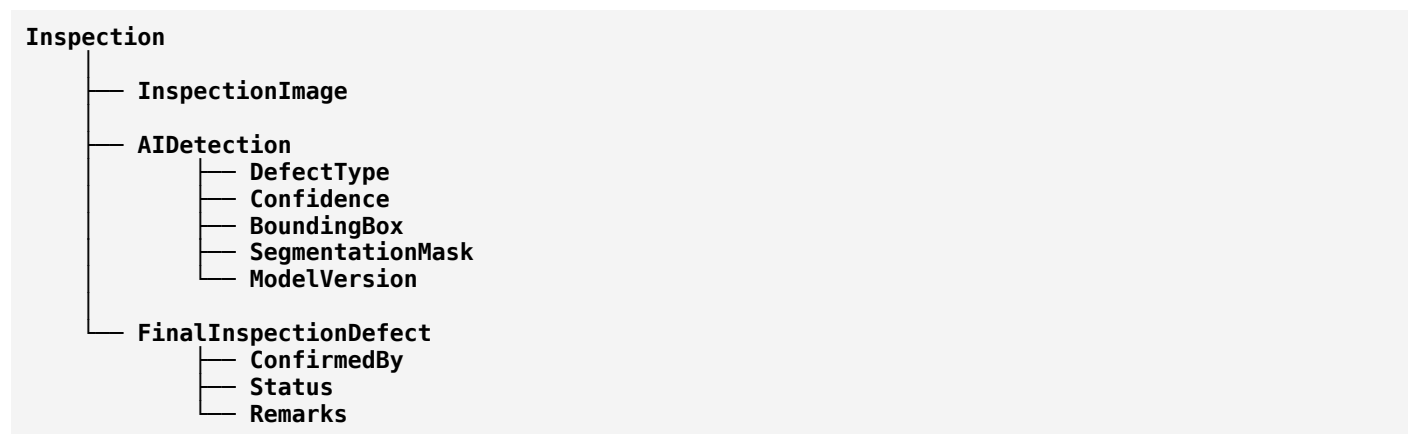
Otherwise, the model may incorrectly detect:

- Shadows
- Light reflections
- Wall texture
- Dirt
- Camera noise

as scratches.

Suggested database design

I would keep the AI detection separate from the final inspector decision.



Example:

InspectionAIDetection

```

SQL
CREATE TABLE InspectionAIDetection
(
    AIDetectionID BIGINT IDENTITY(1,1) PRIMARY KEY,
    InspectionImageID BIGINT NOT NULL,

    DefectTypeID INT NULL,
    DefectType NVARCHAR(100) NULL,

    Confidence DECIMAL(5,4) NOT NULL,

    X DECIMAL(10,4) NULL,
    Y DECIMAL(10,4) NULL,
    Width DECIMAL(10,4) NULL,
    Height DECIMAL(10,4) NULL,

    SegmentationData NVARCHAR(MAX) NULL,

    ModelName NVARCHAR(100) NULL,
    ModelVersion NVARCHAR(100) NULL,

    IsConfirmed BIT NULL,
    ConfirmedBy INT NULL,
    ConfirmedDate DATETIME NULL,

    CreatedDate DATETIME NOT NULL DEFAULT GETDATE()
);
  
```

This gives you an **AI audit trail**, which is valuable for property inspection:

Model detected → Inspector reviewed → Inspector accepted/rejected.

My recommended roadmap

Phase 1 — Dataset

Collect around **500–1,000+ images** initially across your most common defects.

Phase 2 — Annotation

Use Roboflow to annotate:

- Scratch
- Crack
- Stain
- Paint damage
- Hole

Phase 3 — Train

Train a YOLO detection baseline first.

Phase 4 — Evaluate

Measure:

- Precision
- Recall
- False positives
- False negatives

Don't evaluate only by overall mAP; for property inspection, missing a serious defect may be more important than the aggregate metric.

Phase 5 — Segmentation experiment

Train a segmentation model specifically for:

- Scratches
- Cracks
- Paint damage

Phase 6 — .NET integration

Export the selected model to **ONNX** and run inference using **ONNX Runtime**, or expose the model through a Python/FastAPI inference service. Ultralytics documents ONNX export for both detection and segmentation models. [Ultralytics Docs+1](#)

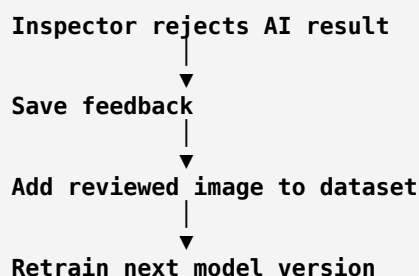
Phase 7 — Human-in-the-loop

React displays:

```
AI detected: Scratch
Confidence: 87%

[✓ Confirm] [✗ Reject] [Edit]
```

Phase 8 — Continuous learning



My final recommendation

For **your property inspection application**, I would use:

Roboflow initially for dataset management and annotation → train YOLO as the baseline → test segmentation for small wall defects → deploy the final selected model into your ASP.NET Core architecture.

For the specific kind of subtle scratch shown in your uploaded image, **segmentation or high-resolution/tiled inference is likely worth testing**, because the defect is small and irregular. I would not expect an off-the-shelf pretrained YOLO model to reliably detect this without a carefully labeled dataset of similar property defects.



In Phase 2 — Annotation, what if I use **Label Studio** for the annotation process?

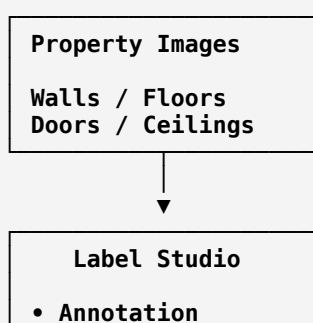
Yes — **Label Studio is a very good choice for your Phase 2 annotation**, especially for your property inspection system.

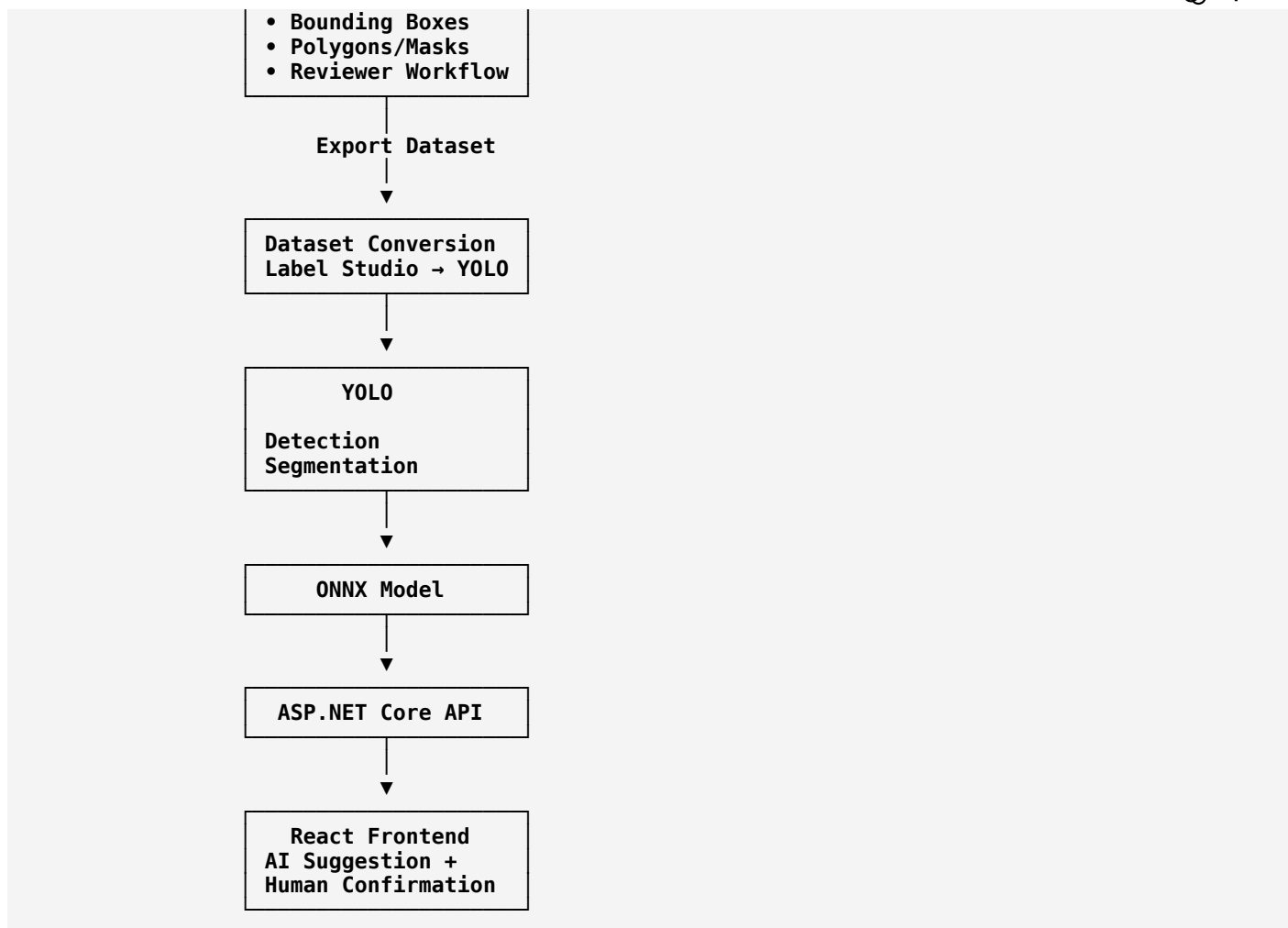
In fact, given your architecture and the need for **inspector-reviewed annotations**, I would seriously consider this stack:

Label Studio → YOLO → ONNX → ASP.NET Core API → React → SQL Server

You don't need Roboflow for annotation if Label Studio handles your workflow well.

Revised architecture

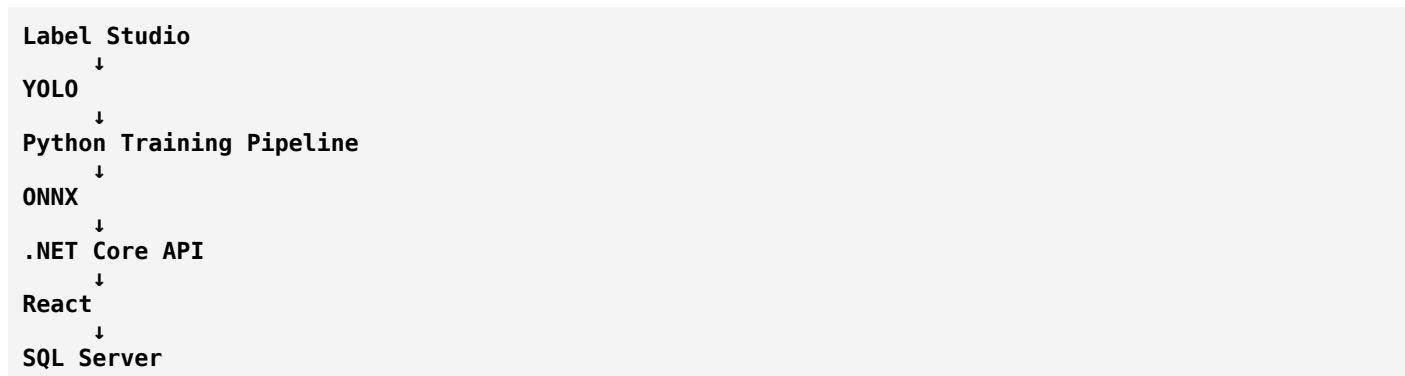




My recommendation for your project

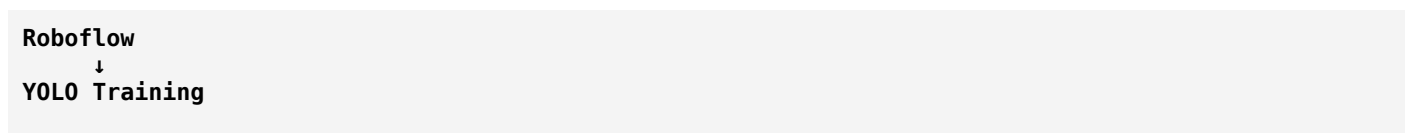
I would actually change the earlier recommendation slightly:

Option A — Fully controlled/self-hosted



This would be my preferred architecture for your enterprise property inspection application.

Option B — Faster managed workflow



Deployment

This is simpler initially, but Label Studio gives you more flexibility and control over the annotation process.

Why Label Studio fits your use case

Your system already has domain experts/inspectors.

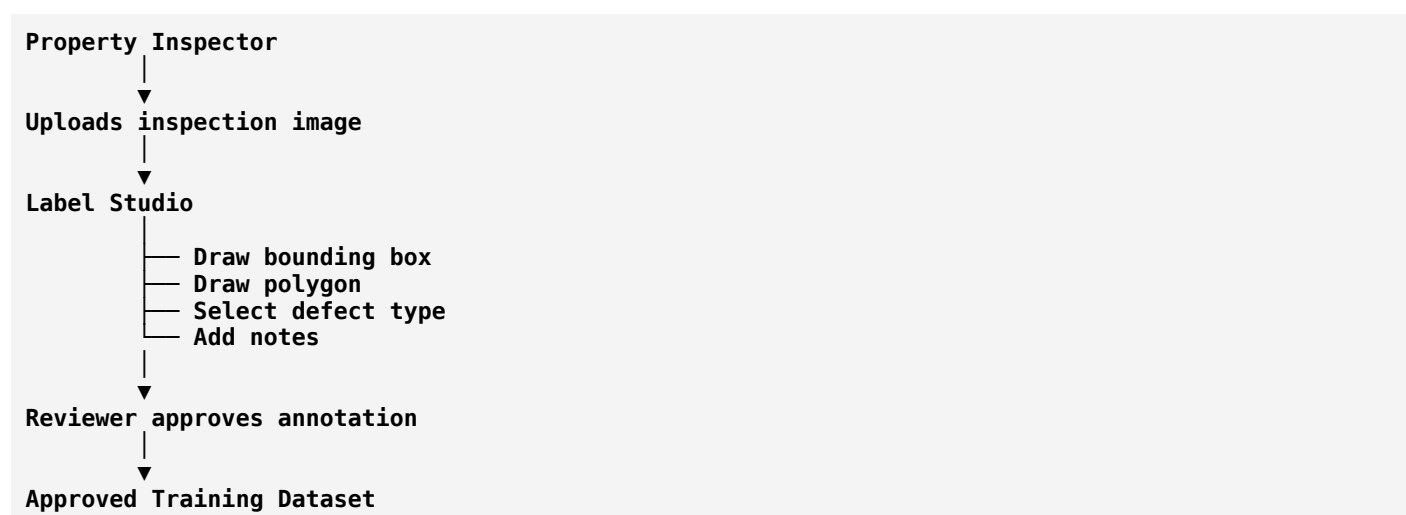
That is important because the people who understand whether something is:

- A scratch
- A crack
- Paint damage
- Normal wall texture
- A stain
- Dampness
- A construction defect

are your **property inspectors**, not ML engineers.

Label Studio can become the interface between your property inspection experts and the AI training pipeline.

Workflow



This is particularly useful because **annotation quality will directly affect your model quality**.

For your wall scratch example

I would define labels such as:

```

Defect
├── Scratch
├── Crack
├── PaintDamage
├── Stain
├── Dampness
├── Hole
└── OtherDamage

```

For a simple first YOLO experiment, use **bounding boxes**.

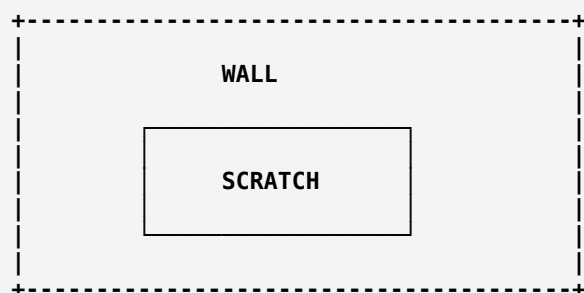
Example Label Studio configuration conceptually:

```

Image
├── Rectangle Labels
│   ├── Scratch
│   ├── Crack
│   ├── Paint Damage
│   ├── Stain
│   ├── Dampness
│   └── Hole

```

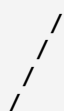
The annotation result would look conceptually like:



But for scratches, consider polygons

For your specific uploaded image, I would eventually move from bounding boxes to **polygon/segmentation annotations**.

The scratch is approximately like this:



A bounding box becomes:



Most of that box is actually **normal wall**.

With segmentation:

Normal Wall



Only the actual defective region is labeled.

Therefore:

Defect	Recommended annotation
Large stain	Bounding box or polygon
Hole	Bounding box
Broken switch	Bounding box
Missing object	Bounding box
Crack	Polygon/segmentation
Small scratch	Polygon/segmentation
Paint damage	Polygon/segmentation
Dampness	Polygon/segmentation

Suggested Label Studio project structure

I would create separate annotation projects initially.

Project 1: Object Defects

Property Inspection - Object Defects

Labels:

BrokenSwitch
MissingCover
BrokenTile
Hole
DamagedDoor
DamagedHandle

Use:

Bounding Boxes → YOLO Detection

Project 2: Surface Defects

Property Inspection - Surface Defects

Labels:

```
Scratch
Crack
PaintDamage
Stain
Dampness
```

Use:

Polygons → YOLO Segmentation

This separation is useful because the characteristics of these defects are very different.

Add annotation metadata

This is something I strongly recommend for your property domain.

Don't only annotate the defect type.

Capture metadata such as:

Defect Type: Scratch

Severity:

- ☐ Minor
- ☐ Medium
- ☐ Major

Location:

- ☐ Wall
- ☐ Floor
- ☐ Ceiling
- ☐ Door

Action:

- ☐ Repair
- ☐ Replace
- ☐ Clean
- ☐ Monitor

For example:

JSON

```
{
  "defectType": "Scratch",
  "severity": "Minor",
  "location": "Wall",
  "recommendedAction": "Repair"
}
```

However, I would **not necessarily train all of these values into your first YOLO model**.

Start with:

IMAGE

↓

AI detects:
Scratch
Crack
Stain

Then let either your inspector or a separate AI/business-rules layer determine:

```
Severity
Responsibility
Repair action
Cost
Priority
```

This keeps the computer vision model simpler.

Dataset versioning is important

Since you have a production property inspection application, don't simply export a dataset and overwrite it every time.

Use versions conceptually like:

```
PropertyDefectDataset
├── v1.0
│   ├── 500 Images
│   ├── 5 Classes
│   └── Initial Training
├── v1.1
│   ├── 750 Images
│   ├── Improved Scratch Labels
│   └── Added Negative Images
└── v2.0
    ├── 2,000 Images
    ├── 8 Classes
    └── Added Segmentation
```

Then associate the model with the dataset version:

```
Model: property-defect-yolo-v1
Dataset: PropertyDefectDataset-v1.0

Model: property-defect-yolo-v2
Dataset: PropertyDefectDataset-v1.1
```

Your SQL Server audit record can store:

```
SQL
ModelName
ModelVersion
DatasetVersion
Confidence
PredictionDate
```

This is important when someone asks:

"Why did the system detect this as a scratch?"

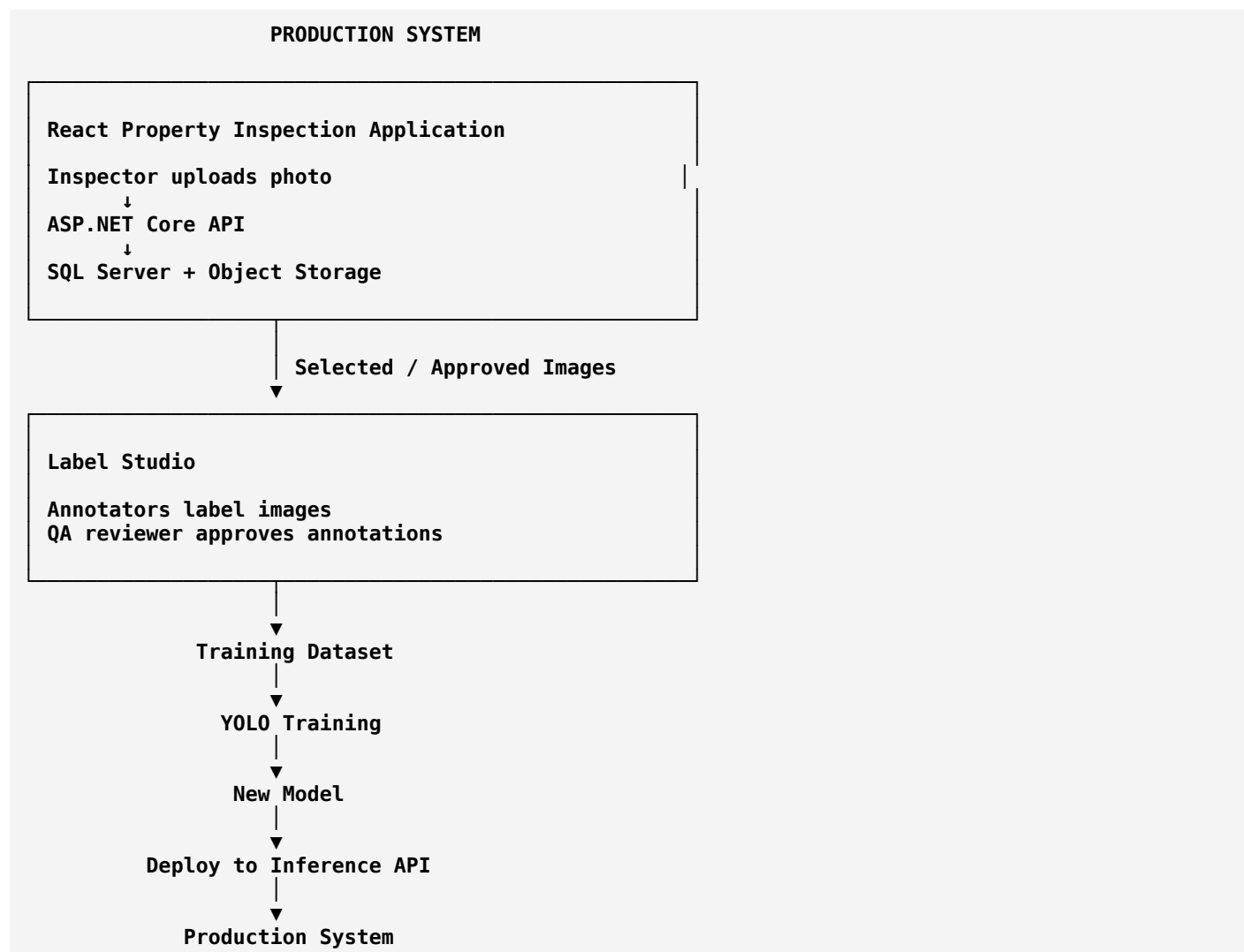
You can trace which model made the prediction.

Recommended integration with your existing application

I would **not make Label Studio your primary property inspection application.**

Your users should continue using your React application.

Use Label Studio as an internal **AI data preparation and annotation tool.**



Very important: use AI-assisted annotation later

Once you have your first trained YOLO model, your process can become much faster.

Initially

```

1,000 Images
↓
Human annotates all images manually
↓
Train YOLO v1

```

After YOLO v1

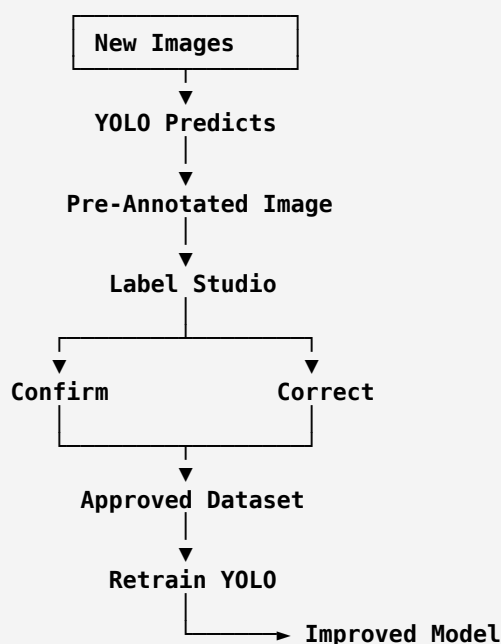
```

New Images
↓
YOLO v1 predicts defects
↓
Pre-label annotations
↓

```

Human inspector reviews
 ↓
 Corrects mistakes
 ↓
 Approved annotations
 ↓
 YOLO v2 training

This is the **human-in-the-loop active learning cycle** I would recommend for your project.



Bottom line

Yes, use Label Studio. For your system, I would recommend:

Label Studio for annotation and review → YOLO for training → ONNX for production inference → ASP.NET Core for integration → React for inspector confirmation → SQL Server for audit/history.

For your **small wall scratches and cracks**, I would start with a YOLO detection baseline to validate the dataset and workflow, then create a **segmentation dataset/model for surface defects** where detection boxes prove too imprecise. This gives you a practical path without over-engineering the first version.



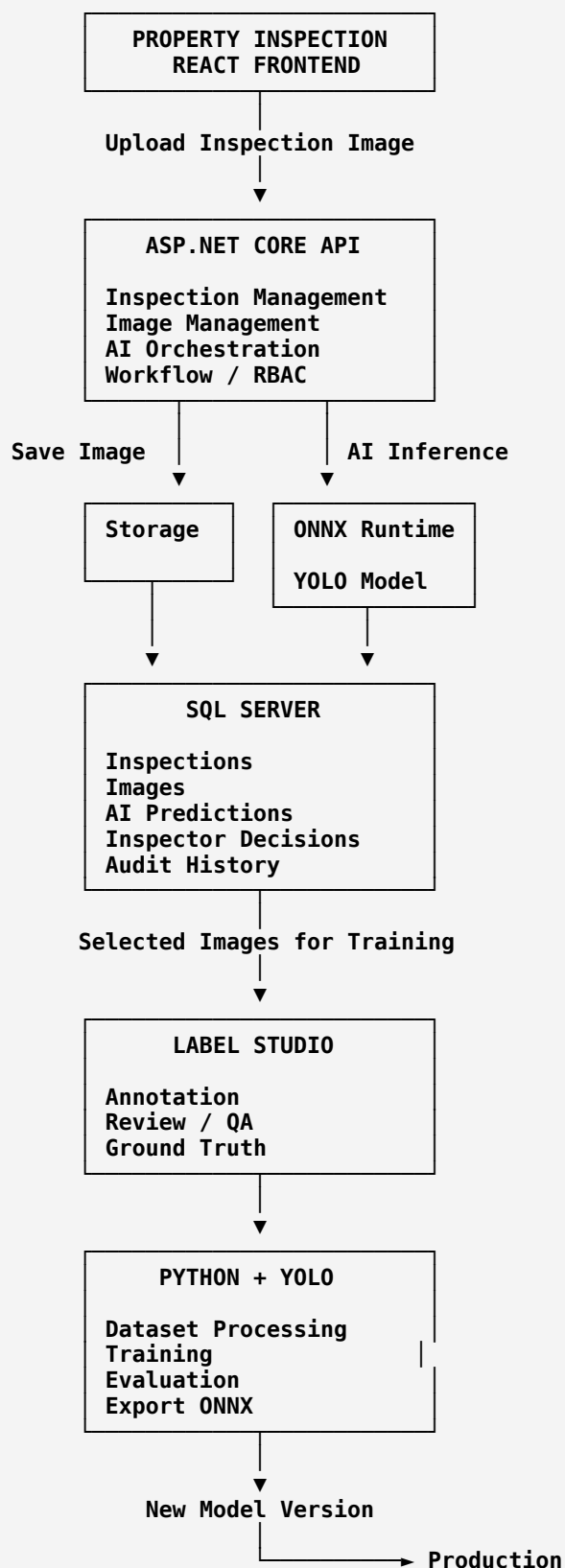
As a senior software engineer can I build the product system, entirely with:

Label Studio for annotation and review → YOLO for training → ONNX for production inference → ASP.NET Core for integration → React for inspector confirmation → SQL Server for audit/history.

Yes. **Absolutely.** As a senior software engineer with your .NET, SQL Server, React, Docker, and AI integration background, you can build this end-to-end.

More importantly, your proposed architecture is a **realistic production architecture**, not just a proof of concept.

Recommended architecture



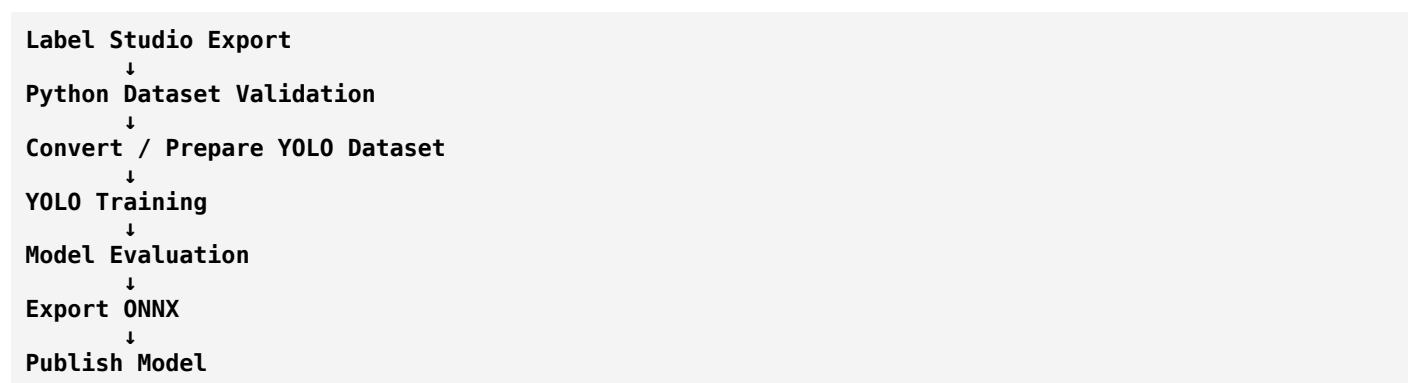
The only important addition: Python

I would **not try to eliminate Python completely**.

Your production application can absolutely be primarily:

- ASP.NET Core
- React
- SQL Server
- ONNX Runtime

But your **ML pipeline** should use Python:



That is not a weakness in your architecture. It is the clean separation of:

Enterprise application engineering and **machine learning engineering**.

What you can build yourself

You do **not** need to become a computer vision researcher before starting.

You can build approximately 80–90% of the product using your existing software engineering skills.

Your existing expertise

Area	Your background	Role in this system
ASP.NET Core	Strong	Core backend
SQL Server	Strong	Data and audit
React	Existing experience	Inspection UI
Docker	Existing experience	Deploy services
REST APIs	Strong	Service integration
System architecture	Strong	Overall platform
Python	Need practical ML skills	Training pipeline
Computer vision	Need domain learning	Model/data decisions
Deep learning theory	Helpful, not initially mandatory	Optimization

Your biggest learning curve is **not writing the software**.

It is understanding:

1. Dataset quality
 2. Annotation consistency
 3. Image preprocessing
 4. Model evaluation
 5. False positives/false negatives
 6. Small-object detection
 7. Model retraining and versioning
-

I would split the product into 5 systems

1. Property Inspection Application

This is your main application.

```
React
  ↓
ASP.NET Core
  ↓
SQL Server
```

Responsibilities:

- Projects
- Properties
- Units
- Inspection scheduling
- Inspectors
- Checklists
- Activities
- Defects
- Images
- Approvals
- Reports

You already have significant experience with this part.

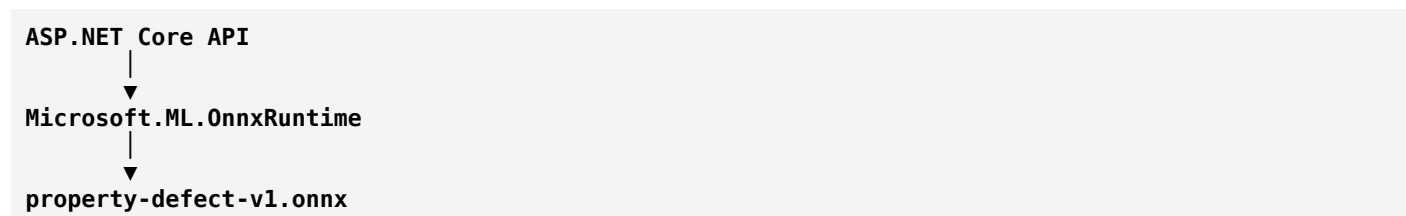
2. AI Inference Service

This is responsible for answering:

"What defects can AI see in this image?"

You have two good options.

Option A — ONNX directly inside ASP.NET Core



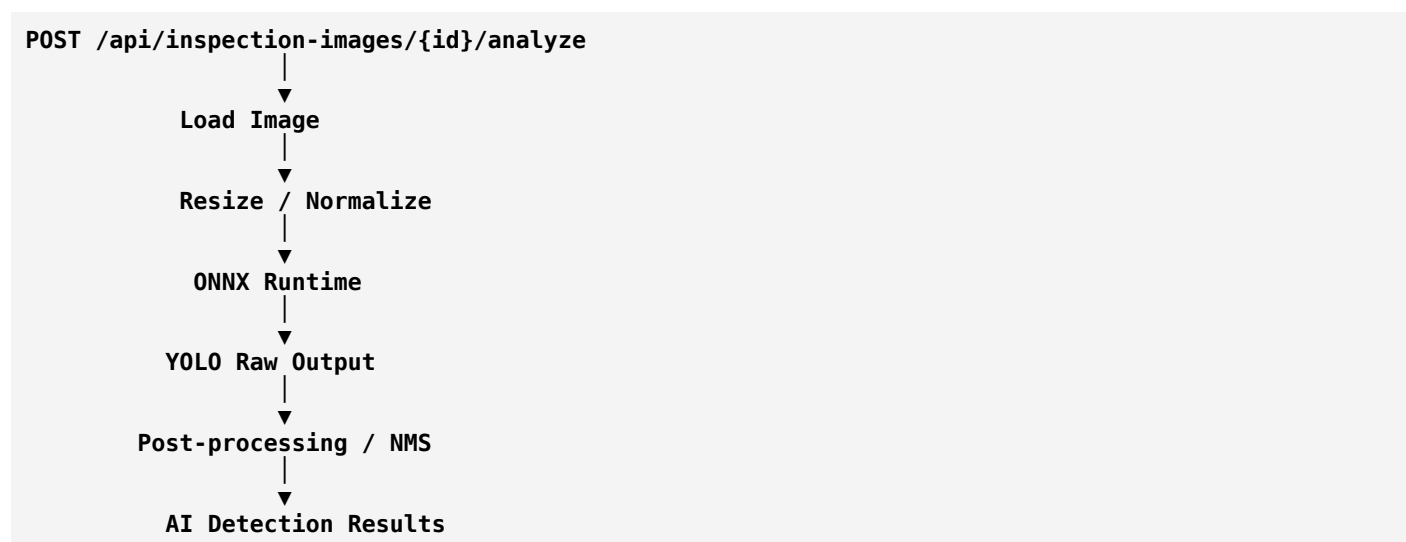
For your architecture, I would start here.

```

C#
using Microsoft.ML.OnnxRuntime;

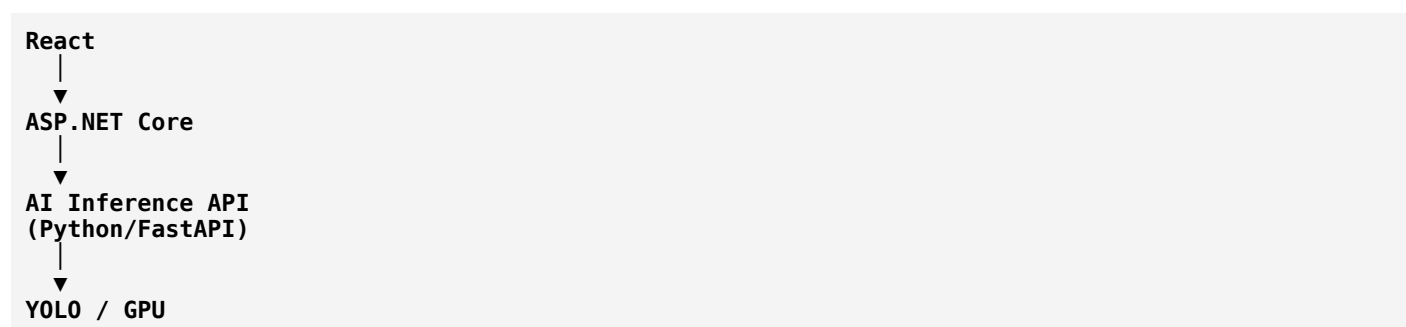
var session = new InferenceSession(
    "Models/property-defect-v1.onnx"
);
  
```

Your flow becomes:



Option B — Separate inference service

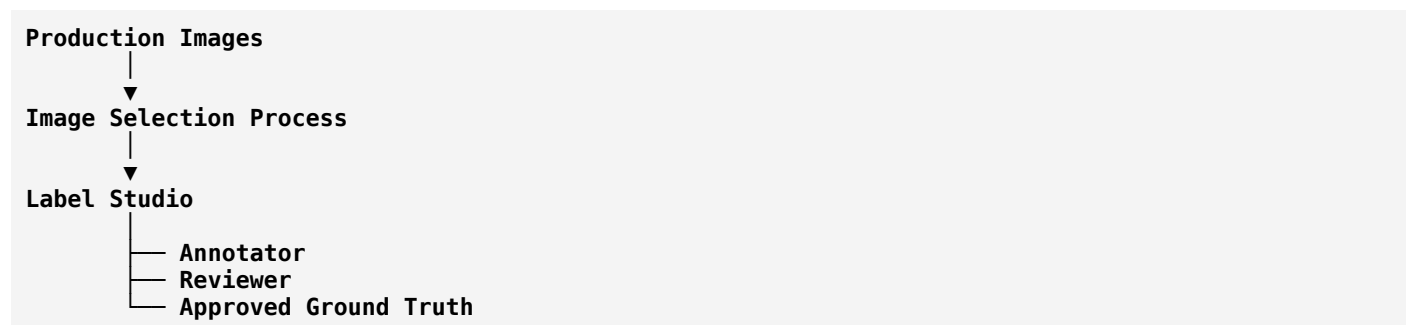
Later, if load grows:



I would **not start here unless you need GPU inference or expect heavy image volume.**

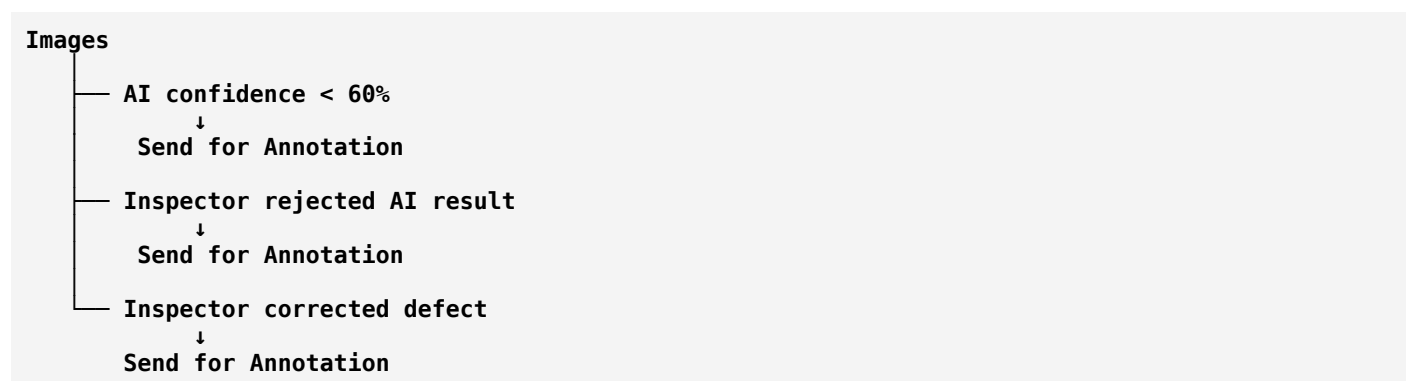
3. Annotation System

Keep Label Studio separate from your production application.



A very important product feature you can build is an **AI Dataset Queue**.

For example:

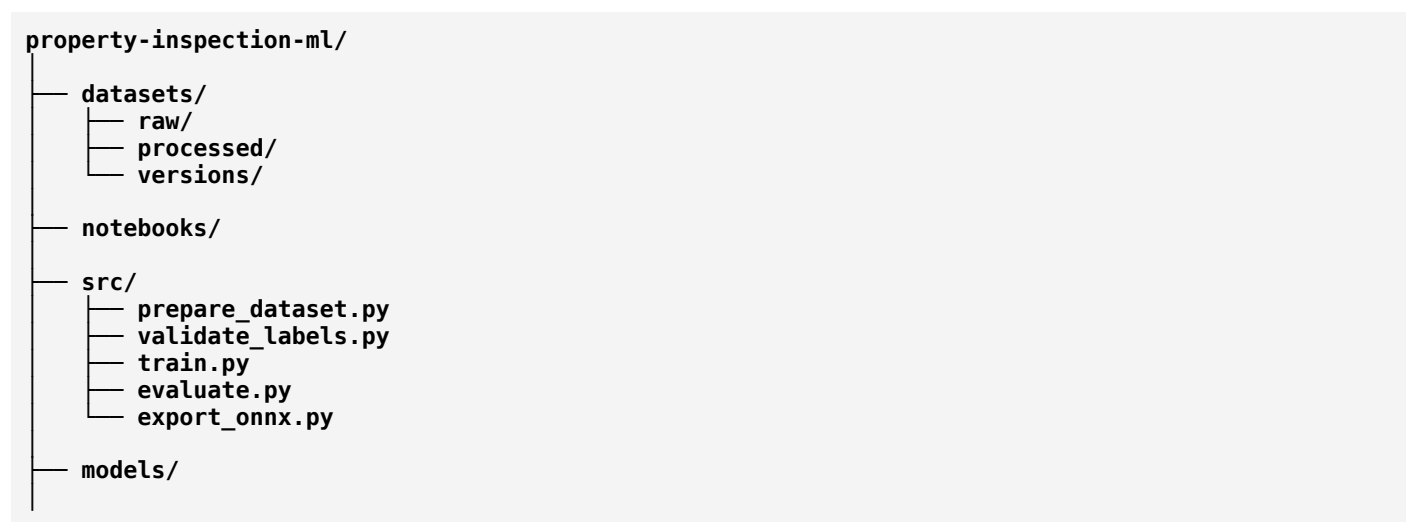


This means your actual product continuously produces valuable training data.

That is a major advantage.

4. ML Training Pipeline

Create this as a separate repository or module:



```

├── configs/
└── requirements.txt

```

The lifecycle:

```

Dataset v1
  ↓
Train YOLO v1
  ↓
Evaluate
  ↓
ONNX Export
  ↓
Deploy
  ↓
Collect Inspector Feedback
  ↓
Dataset v2
  ↓
Train YOLO v2

```

This should be reproducible. You should always be able to answer:

Which dataset produced Model v1.4?

5. Model Registry and Governance

This is where your enterprise software engineering experience becomes particularly valuable.

I would create a table such as:

```

SQL
CREATE TABLE AIModel
(
    AIModelID INT IDENTITY(1,1) PRIMARY KEY,

    ModelName NVARCHAR(200) NOT NULL,
    ModelVersion NVARCHAR(50) NOT NULL,

    ModelType NVARCHAR(50) NOT NULL,
    -- Detection / Segmentation

    DatasetVersion NVARCHAR(50) NOT NULL,

    PrecisionScore DECIMAL(10,6) NULL,
    RecallScore DECIMAL(10,6) NULL,
    MAP50 DECIMAL(10,6) NULL,

    ONNXFilePath NVARCHAR(500) NULL,

    IsActive BIT NOT NULL DEFAULT 0,

    CreatedDate DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CreatedBy INT NULL
);

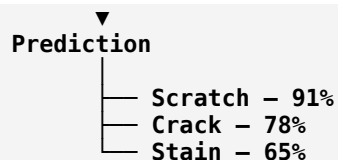
```

Then every prediction records:

```

InspectionImage
  |
  ▼
AIModel v1.4
  |

```

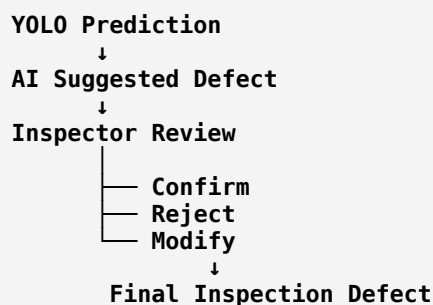


This gives you traceability.

The most important architecture decision

Do **not** directly convert AI predictions into final inspection defects.

Instead:



For example:

AI DETECTION

Defect: Scratch
Confidence: 87%

[Confirm] [Reject] [Edit]

This is exactly where your existing property inspection workflow should remain the **source of truth**.

AI is an assistant, not the final authority.

What I would build first

Don't start with the complete platform.

Build a **vertical slice**.

Sprint 1: One defect

Choose only:

Wall Scratch

Build:

```

100-300 Images
↓
Label Studio Annotation
↓
YOLO Training
↓
Evaluate Model
↓
Export ONNX
↓
ASP.NET Core API
↓
React shows prediction
↓
Inspector Confirm/Reject

```

End-to-end.

Even if the first model is only moderately accurate, you will learn the complete architecture.

Sprint 2

Add:

```

Scratch
Crack
Paint Damage
Stain

```

Then compare:

```

YOLO Detection
  VS
YOLO Segmentation

```

Especially for your small wall scratches.

Sprint 3

Add the feedback loop:

```

Inspector Rejects AI
↓
Save Correction
↓
Training Candidate Queue
↓
Label Studio
↓
Reviewer Approval

```

At this point, you have the foundation of an actual **AI-assisted property inspection platform**.

My strongest recommendation

Yes, build it yourself, but think of it as **two connected products**:

Product A — Inspection Platform

React + ASP.NET Core + SQL Server

Product B — Computer Vision Platform

Label Studio + Python + YOLO + ONNX

Your backend becomes the orchestration layer between them.

Given your existing experience with .NET APIs, SQL Server, Docker, and your previous work exploring YOLO, Qdrant, Ollama, and AI agents, this is a very achievable progression. The key is to **start with one defect type and one complete end-to-end workflow**, rather than trying to solve every property defect from day one.

Today 3:49 PM



summaries this and create a complete production grade flow as well as a senior software enigneer write a prompt for this to step by step implement this.

Below is a consolidated **production-grade blueprint** for your AI-assisted Property Inspection System, followed by a **master implementation prompt** you can use step by step.



AI-Assisted Property Inspection System — Production Architecture & Implementation Blueprint

1. Executive Summary

The proposed system combines traditional property inspection software with computer vision to help inspectors detect defects from uploaded inspection images.

The architecture consists of two connected domains:

A. Property Inspection Platform

Responsible for business operations and remains the system of record.

- React frontend
- ASP.NET Core Web API
- SQL Server
- Image/object storage
- Authentication and RBAC

- Inspection workflows
- Defect management
- Inspector review and approval
- Audit history and reporting

B. AI / Computer Vision Platform

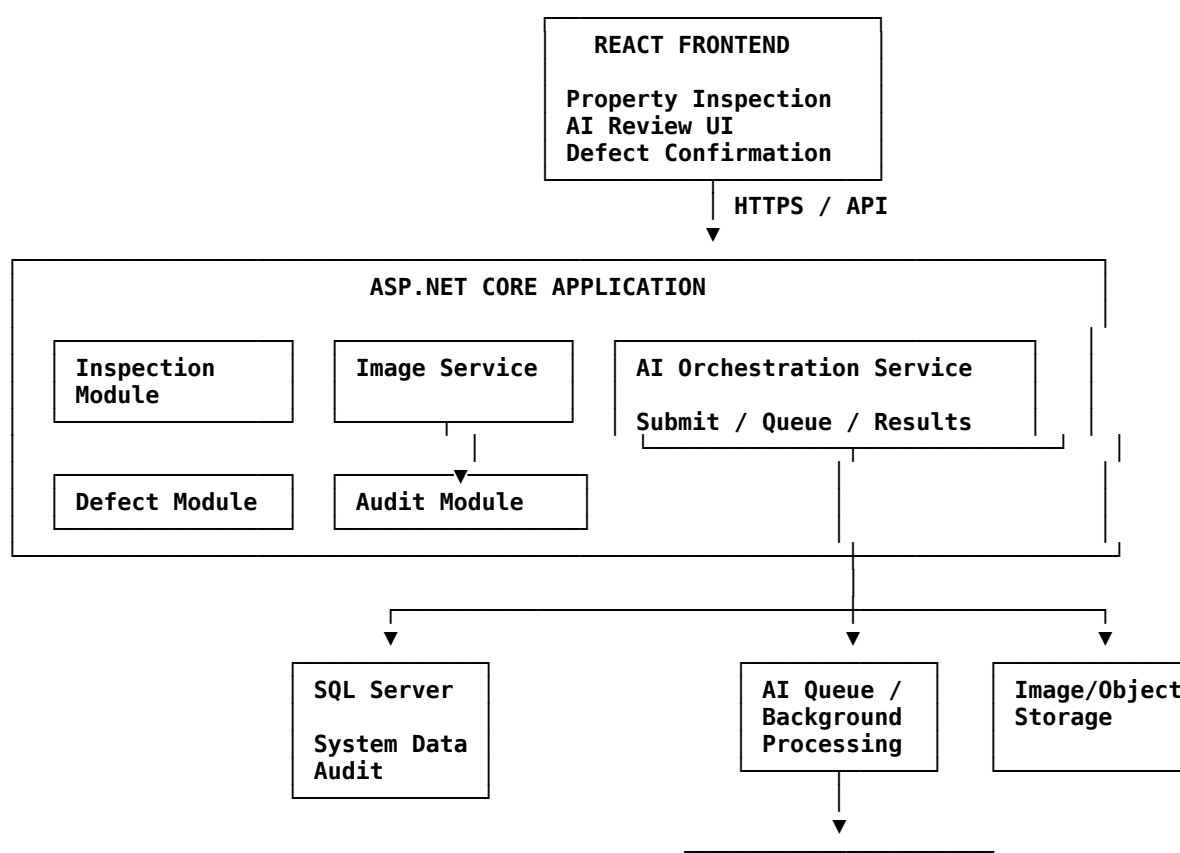
Responsible for learning from inspection images and generating AI predictions.

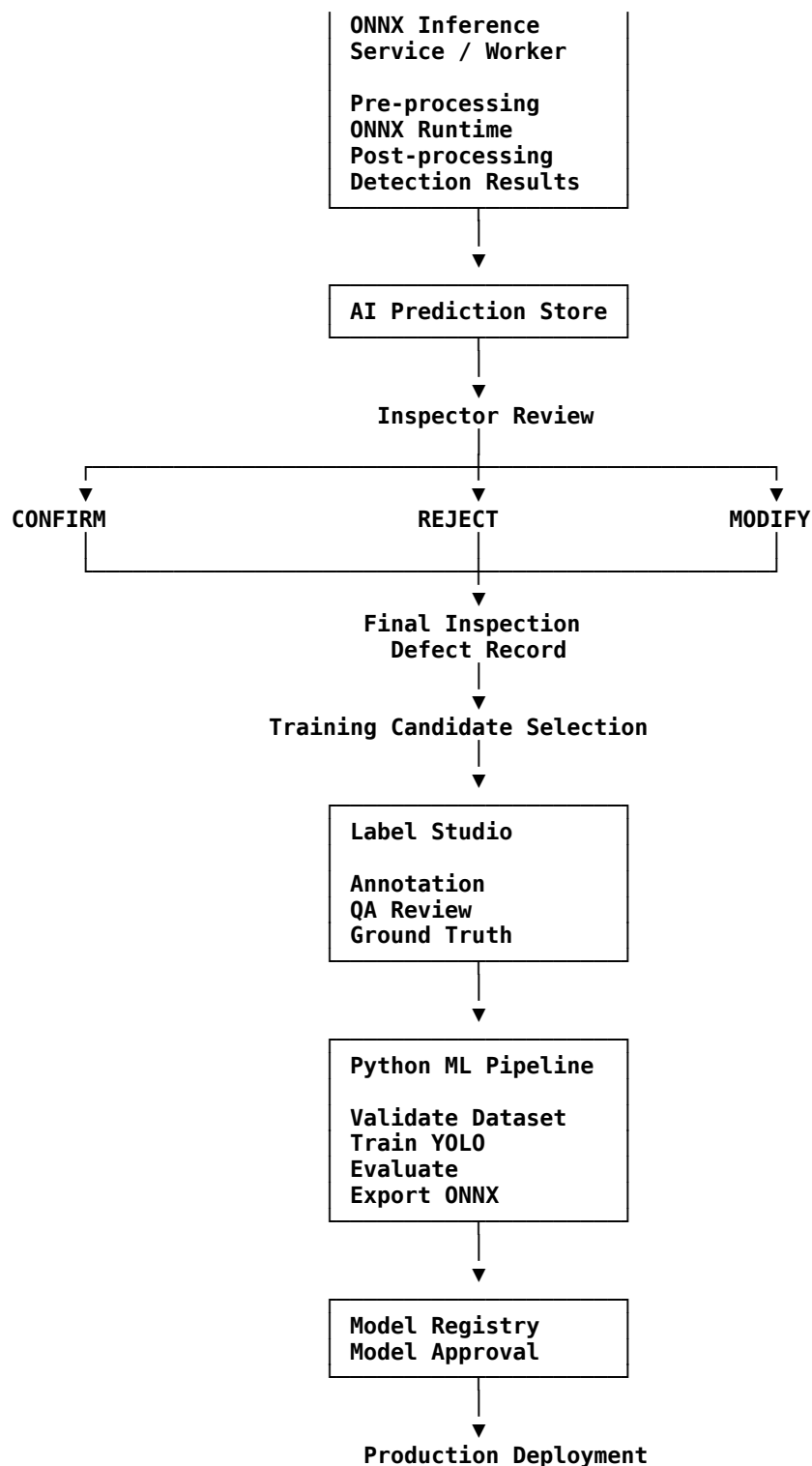
- Label Studio for annotation and review
- Python for dataset preparation and ML orchestration
- YOLO for object detection and segmentation training
- ONNX as the production model format
- ONNX Runtime for inference
- Model versioning and evaluation
- Continuous feedback and retraining

The most important architectural principle is:

AI predictions are suggestions. Human inspectors remain responsible for confirming, rejecting, or modifying the final inspection defect.

2. Target Production Architecture



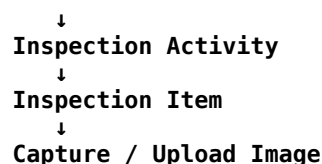


3. End-to-End Production Flow

Phase 1 — Inspection Image Capture

The inspector performs a normal property inspection.

Property
↓
Unit
↓
Workspace / Area



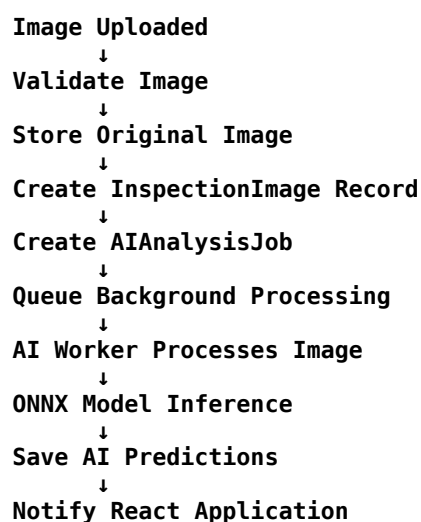
The React application uploads the image to the ASP.NET Core API.

The backend:

1. Validates the file.
2. Validates size and format.
3. Generates a unique image identifier.
4. Stores the original image in object storage.
5. Stores metadata in SQL Server.
6. Creates an AI analysis request.

The original image should never be modified.

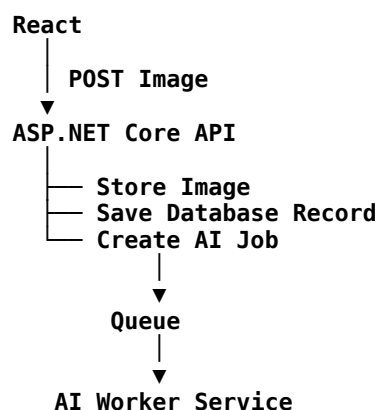
4. Image Processing Flow

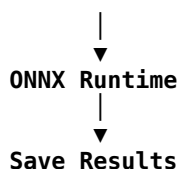


Do not run expensive AI inference synchronously inside the image upload request.

Use asynchronous processing for production workloads.

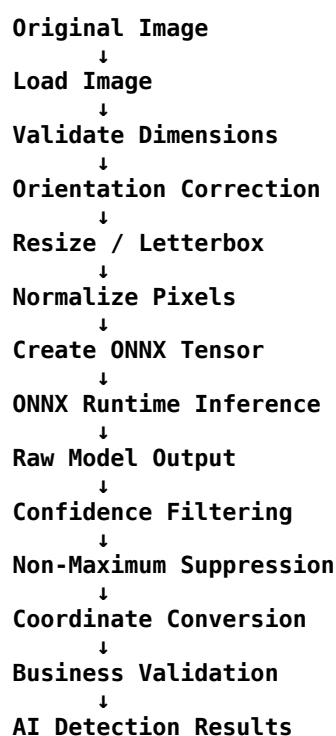
Recommended pattern:



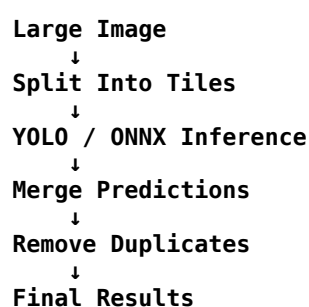


5. AI Inference Flow

The inference pipeline should have clear stages.



For small defects such as scratches and cracks, the architecture should support future tiled or high-resolution inference.

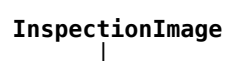


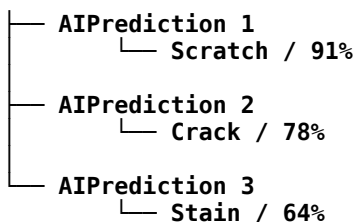
This should be implemented as an extensible inference strategy rather than hard-coded into the first version.

6. AI Prediction Model

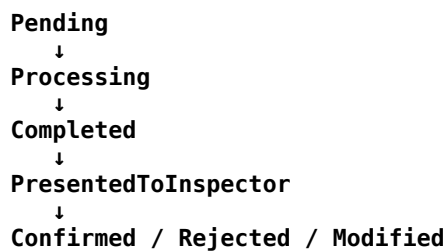
AI predictions should be stored separately from confirmed inspection defects.

Example:





Recommended lifecycle:



An AI prediction should contain:

- Prediction ID
- Inspection Image ID
- Model ID
- Model version
- Dataset version
- Defect type
- Confidence
- Bounding box
- Polygon or mask when applicable
- Processing timestamp
- Review status
- Inspector feedback

7. Inspector Confirmation Flow

The React frontend displays AI findings as suggestions.

AI SUGGESTED DEFECT

Type: Scratch

Confidence: 87%

[CONFIRM]

[REJECT]

[MODIFY]

Possible actions:

Confirm

```

AI Prediction
  ↓
Inspector Confirms
  ↓
Final Defect Created
  ↓
Prediction Status = Confirmed

```

Reject

```

AI Prediction
  ↓
Inspector Rejects
  ↓
Prediction Status = Rejected
  ↓
Potential Training Candidate

```

Modify

```

AI Prediction
  ↓
Inspector Changes:
- Defect type
- Coordinates
- Severity
  ↓
Final Defect Created
  ↓
Original AI Prediction Preserved
  ↓
Potential Training Candidate

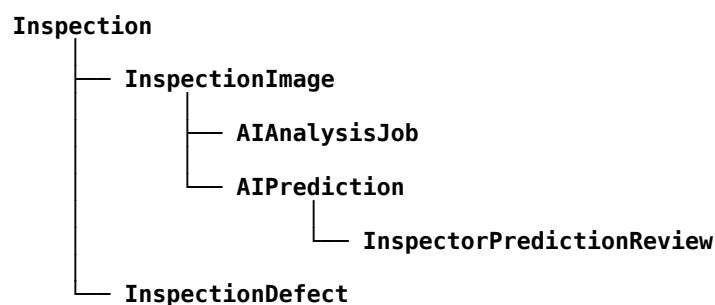
```

Never overwrite the original AI prediction.

This provides a complete audit trail.

8. SQL Server Logical Data Model

Core entities

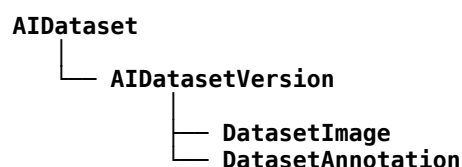


AI Model entities



└─ AIModelDeployment

Dataset entities



Recommended tables

AIModel
AIModelVersion
AIModelDeployment

AIAnalysisJob
AIPrediction
AIPredictionReview

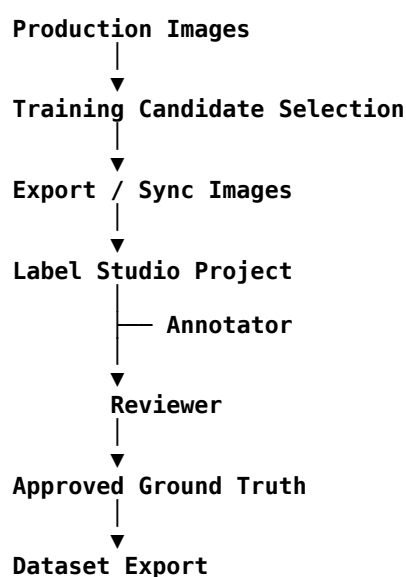
AIDataset
AIDatasetVersion
AITrainingCandidate

AITrainingRun
AIModelMetric

AIModelDeploymentHistory

9. Annotation and Ground Truth Flow

Label Studio should be treated as an internal annotation platform, not the main production application.



Recommended candidate sources:

1. Inspector rejected AI prediction
2. Inspector modified AI prediction
3. Low-confidence prediction
4. High-confidence incorrect prediction

5. New property/environment type
6. New defect type
7. Random quality sampling

This creates a continuous improvement cycle.

10. Annotation Strategy

Use separate projects or clear schemas for different computer vision problems.

Object Defects

Examples:

- Broken switch
- Missing cover
- Broken tile
- Hole
- Damaged handle

Recommended model:

YOLO Object Detection

Recommended annotation:

Bounding Boxes

Surface Defects

Examples:

- Scratch
- Crack
- Paint damage
- Dampness
- Stain

Recommended model:

YOLO Segmentation

Recommended annotation:

Polygon / Mask

Start with a smaller set of high-frequency defects.

Recommended initial classes:

Scratch
Crack
PaintDamage
Stain
Dampness
Hole

Do not attempt to solve every possible property defect in version one.

11. Dataset Quality Pipeline

Before training, every dataset version should pass validation.

```

Label Studio Export
↓
Schema Validation
↓
Image Exists Validation
↓
Label Validation
↓
Bounding Box / Polygon Validation
↓
Duplicate Detection
↓
Corrupt Image Detection
↓
Class Distribution Analysis
↓
Train / Validation / Test Split
↓
Approved Dataset Version
  
```

The training pipeline should fail if critical validation rules fail.

Examples:

Invalid class name
Missing image
Zero-size bounding box
Invalid polygon
Duplicate image
Corrupted image
Class not supported
Annotation outside image boundaries

12. YOLO Training Pipeline

```

Dataset Version
↓
Dataset Validation
↓
Dataset Conversion
↓
Train YOLO Model
↓
Validate Model
↓
Evaluate Test Set
↓
Generate Metrics
↓
Compare Against Baseline
↓
Model Approval Gate
  
```

↓
Export ONNX

Every training run must record:

- Training run ID
- Dataset version
- Model architecture
- Hyperparameters
- Training date
- Random seed
- Metrics
- Generated model artifacts

Example:

Dataset: PropertyDefectDataset-v1.2
Model: YOLO-Segmentation
Training Run: 2026-0015

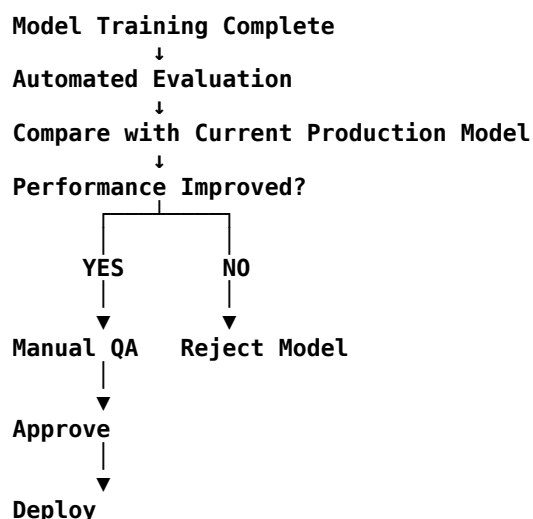
Precision: X
Recall: X
mAP50: X
mAP50-95: X

Artifact:
 property-defect-v1.2.onnx

13. Model Evaluation and Release Gates

Do not deploy a model automatically simply because training completed.

Use a release gate.



Important metrics:

- Precision
- Recall

- mAP50
- mAP50-95
- Per-class precision
- Per-class recall
- False positive rate
- False negative rate
- Inference latency

Evaluate per defect type.

Do not rely only on one overall mAP score.

A model may be excellent at detecting stains but poor at detecting scratches.

14. ONNX Production Deployment

The production model should be versioned.

```
models/
├── property-defect-v1.0/
│   ├── model.onnx
│   └── metadata.json
├── property-defect-v1.1/
│   ├── model.onnx
│   └── metadata.json
└── property-defect-v2.0/
    ├── model.onnx
    └── metadata.json
```

The metadata should include:

```
{
  "modelName": "property-defect",
  "version": "1.2.0",
  "modelType": "segmentation",
  "datasetVersion": "1.4.0",
  "classes": [
    "Scratch",
    "Crack",
    "PaintDamage",
    "Stain"
  ]
}
```

The application should load only an approved active model.

Recommended deployment process:

```
Model Registered
  ↓
Staging Deployment
  ↓
Smoke Tests
  ↓
Inference Validation
  ↓
Production Approval
  ↓
Activate New Model
```

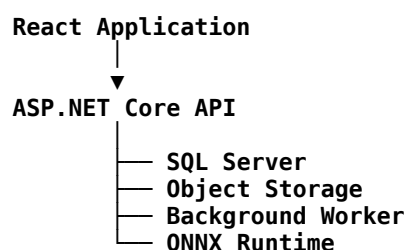
↓
Monitor
↓
Rollback if Required

Never replace a production model file manually without recording the deployment.

15. Recommended Service Architecture

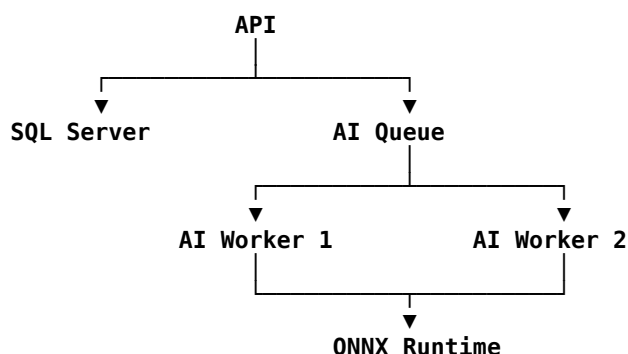
Start with a modular architecture rather than unnecessary microservices.

Initial production architecture



This is simpler to operate.

Scale when required



Only split the AI inference service into a dedicated service when there is a genuine scaling, GPU, deployment, or dependency reason.

16. Security and Production Requirements

The production system should include:

Image security

- File type validation
- File size limits
- Malware scanning where required

- Private object storage
- Signed or authorized image access
- Original file preservation
- Image checksum

API security

- JWT or existing enterprise authentication
- Role-based authorization
- Inspector permissions
- Reviewer permissions
- AI administrator permissions

Audit

Record:

- Who uploaded the image
- Which model analyzed it
- Model version
- Original prediction
- Who confirmed/rejected it
- What was modified
- Timestamp of every action

Data retention

Define policies for:

- Original inspection images
- Training copies
- Rejected predictions
- Annotation datasets
- Old models

17. Observability

Monitor the complete pipeline.

Application metrics

- Images uploaded
- AI jobs created
- AI jobs completed
- AI job failures
- Average processing time

Model metrics

- Prediction confidence distribution
- Predictions per class
- Inspector confirmation rate
- Inspector rejection rate
- Modification rate
- False-positive indicators

Infrastructure metrics

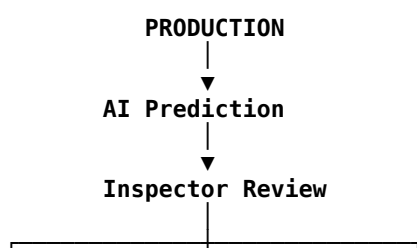
- CPU
- Memory
- GPU when applicable
- Queue depth
- ONNX inference latency

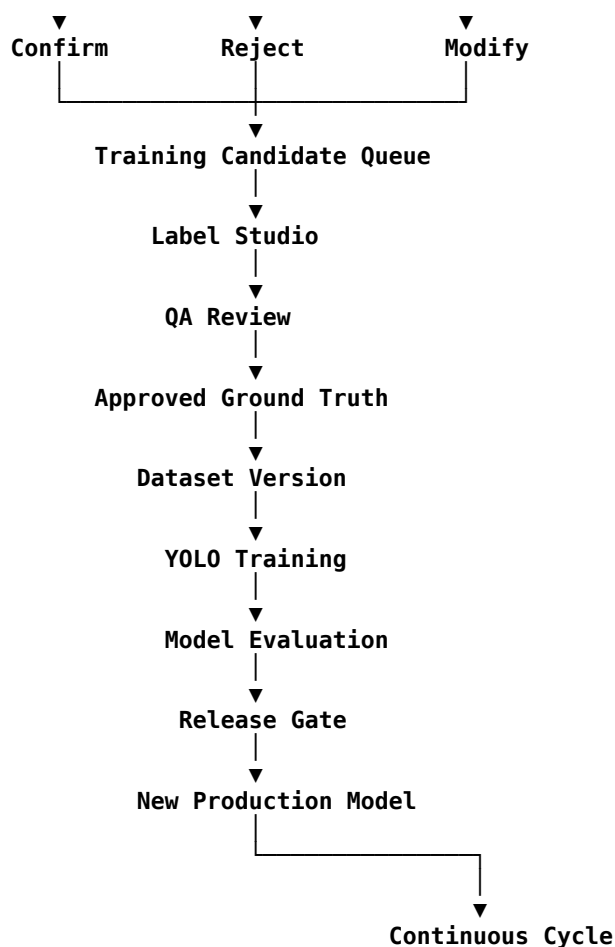
Recommended alerts:

AI job failure rate > threshold
 Queue backlog too high
 Model loading failure
 Inference latency too high
 Storage unavailable

18. Continuous Learning Flow

This is the core long-term AI improvement process.





19. Recommended Implementation Phases

Phase 1 — Foundation

Build the existing inspection integration.

Implement:

- Inspection image entity
- Object storage
- Image upload API
- Image metadata
- Background job architecture
- Audit logging

Do not add AI yet.

Phase 2 — Annotation Platform

Implement Label Studio.

Create:

- Surface defect project
- Object defect project
- Annotation instructions
- Reviewer process
- Dataset export process

Start with 100–300 images for a proof of concept.

Phase 3 — First ML Pipeline

Build Python scripts for:

- Dataset validation
- Dataset conversion
- YOLO configuration
- Training
- Evaluation
- ONNX export

Start with one defect class:

Scratch

The goal is learning the complete end-to-end pipeline, not achieving perfect accuracy.

Phase 4 — Production Inference

Integrate ONNX Runtime into ASP.NET Core.

Implement:

```

Image
↓
Pre-processing
↓
ONNX
↓
Post-processing
↓
AIPrediction

```

Store the complete result in SQL Server.

Phase 5 — Inspector Review

Build React components for:

- AI overlays
- Bounding boxes
- Confidence display
- Confirm
- Reject
- Modify

Ensure the original AI prediction remains immutable.

Phase 6 — Feedback Loop

Implement:

```

Rejected Prediction
  ↓
Modified Prediction
  ↓
Low Confidence Result
  ↓
Training Candidate Queue
  
```

Create synchronization or export from the candidate queue to Label Studio.

Phase 7 — Model Governance

Implement:

- Dataset versions
- Training runs
- Model versions
- Metrics
- Approval
- Deployment
- Rollback

Phase 8 — Scale and Optimization

Add only when required:

- Multiple inference workers
- Queue infrastructure

- GPU inference
- Tiled inference
- Model caching
- Load testing
- Autoscaling

20. Recommended Repository Structure

```

property-inspection/
├── frontend/
│   └── React Application
├── backend/
│   ├── PropertyInspection.API
│   ├── PropertyInspection.Application
│   ├── PropertyInspection.Domain
│   ├── PropertyInspection.Infrastructure
│   └── PropertyInspection.AI.Worker
├── ml/
│   ├── src/
│   │   ├── prepare_dataset.py
│   │   ├── validate_dataset.py
│   │   ├── train.py
│   │   ├── evaluate.py
│   │   └── export_onnx.py
│   ├── configs/
│   ├── scripts/
│   └── requirements.txt
├── label-studio/
│   ├── projects/
│   └── labeling-guidelines/
├── infrastructure/
│   ├── docker/
│   └── docker-compose/
└── docs/
    ├── architecture/
    ├── api/
    ├── ai/
    └── deployment/
  
```

21. Master Step-by-Step Implementation Prompt

Use the following prompt as the implementation contract for an AI coding assistant.

Role

You are acting as a Principal Software Architect, Senior ASP.NET Core Engineer, React Engineer, Computer Vision Engineer, DevOps Engineer, and MLOps Engineer.

We are building a production-grade AI-assisted Property Inspection System.

The existing enterprise technology stack is:

- ASP.NET Core Web API
- .NET
- React with TypeScript
- Microsoft SQL Server
- Docker
- Existing property inspection and handover domain

The AI technology stack is:

- Label Studio for annotation and review
- Python for machine learning pipelines
- YOLO for detection and segmentation
- ONNX for production model artifacts
- ONNX Runtime for inference
- Object storage for inspection images

The system must support a human-in-the-loop workflow.

The AI must never automatically become the source of truth for a property defect. Every AI prediction must be reviewed by an authorized inspector or user before becoming a confirmed inspection defect.

Architecture Principles

Follow these rules:

1. Use clean architecture and clear separation of Domain, Application, Infrastructure, and API concerns.
2. Keep the property inspection application as the system of record.
3. Store AI predictions separately from confirmed inspection defects.
4. Preserve original AI predictions and never overwrite them after inspector review.
5. Every prediction must contain model version information.
6. Every model must be traceable to a dataset version and training run.
7. AI inference must be asynchronous and must not block image upload requests.
8. Use background processing and a queue abstraction.
9. Design for horizontal scaling.
10. Do not introduce microservices unless there is a concrete scaling or operational reason.
11. Use database transactions where required.
12. Add structured logging, health checks, error handling, and audit trails.
13. Use secure file handling and authorization.

14. Make all infrastructure configuration environment-based.
15. Write production-quality code, not simplified demo code.

Implementation Rules

Implement the system incrementally.

Do not generate the complete system in one response.

For each phase:

1. Explain the objective.
2. Show the architecture decision.
3. List prerequisites.
4. Create or update the project structure.
5. Implement the database schema.
6. Implement backend code.
7. Implement API contracts.
8. Implement React integration where applicable.
9. Implement tests.
10. Explain how to run locally.
11. Explain how to validate the implementation.
12. Stop and wait for the next phase.

Do not skip implementation details.

When generating code:

- Provide complete files where practical.
- Clearly identify file paths.
- Do not use placeholder business logic for core workflows.
- Do not silently assume missing database tables or services.
- Reuse existing domain entities where integration is possible.
- Explain all schema assumptions before creating tables.
- Use `async/await` correctly.
- Include cancellation tokens where appropriate.
- Use dependency injection.
- Avoid static service dependencies.
- Follow SOLID principles without unnecessary abstraction.

Phase 1 — Architecture and Existing System Integration

First, analyze the existing Property Inspection domain and identify how AI capabilities should integrate with:

- Inspections
- Workspaces
- Activities
- Inspection items
- Existing defects
- Image attachments
- Users and inspectors
- Existing audit history

Do not modify existing tables unnecessarily.

Create an extension architecture around the existing domain.

Design and implement:

- InspectionImage
- AIAalysisJob
- AIPrediction
- AIPredictionReview
- AIModel
- AIModelVersion
- AITrainingCandidate

Create SQL Server tables, indexes, foreign keys, and status lookup strategies.

Provide migration scripts suitable for SQL Server.

Before proceeding to Phase 2, show the complete entity relationship design and explain the data lifecycle.

Phase 2 — Image Upload and Storage

Implement a production-grade image upload pipeline.

Requirements:

- Validate content type.
- Validate file extension.
- Validate maximum file size.
- Generate unique storage keys.
- Preserve original files.
- Store image metadata.

- Calculate checksum.
- Support object storage through an abstraction.
- Prevent unauthorized image access.
- Record audit information.

Create:

- Storage abstraction
- Local development storage implementation
- Production object storage abstraction
- Image upload API
- Image metadata persistence

Do not implement AI inference yet.

Add integration and unit tests.

Phase 3 — Asynchronous AI Analysis Jobs

Implement asynchronous AI processing.

Flow:

Image Uploaded

→ AIAnalysisJob Created

→ Background Worker Claims Job

→ Image Processed

→ Result Stored

Requirements:

- Job status lifecycle
- Retry policy
- Maximum retry count
- Failure tracking
- Idempotency
- Cancellation
- Structured logging
- Correlation IDs

Do not expose raw ONNX logic directly from controllers.

Implement a clean AI analysis service abstraction.

Phase 4 — Label Studio Integration

Design Label Studio as an internal annotation platform.

Implement the process for:

- Selecting training candidates
- Exporting approved images
- Synchronizing image metadata
- Tracking Label Studio task references
- Importing completed annotations
- Tracking annotation review status

Create an adapter layer so the core application is not tightly coupled to Label Studio APIs.

Define the annotation schema for:

Surface defects:

- Scratch
- Crack
- PaintDamage
- Stain
- Dampness

Object defects:

- Hole
- BrokenSwitch
- MissingCover
- BrokenTile

Create version-controlled annotation guidelines.

Phase 5 — Python ML Pipeline

Create a separate ML module.

Implement:

- Dataset validation
- Annotation validation
- Dataset conversion
- Train/validation/test splitting
- Class distribution reporting
- Duplicate detection where feasible
- YOLO training

- Evaluation
- ONNX export

Create reproducible configuration files.

Every training run must record:

- Dataset version
- Model architecture
- Training parameters
- Metrics
- Artifact locations

Start with a single proof-of-concept defect class, Scratch.

After the end-to-end flow works, expand to additional classes.

Phase 6 — ONNX Runtime Inference

Implement production inference using ONNX Runtime.

Create:

- Model loader abstraction
- Model metadata validation
- Image pre-processing pipeline
- Tensor generation
- ONNX execution
- YOLO output parser
- Confidence filtering
- Non-Maximum Suppression
- Coordinate conversion
- Prediction persistence

The architecture must support both detection and segmentation models.

Do not hard-code model class names into inference logic.

Classes must come from model metadata.

Phase 7 — React AI Review Experience

Implement a React TypeScript user experience for AI-assisted inspection.

Requirements:

- Display inspection image.
- Render AI bounding boxes.
- Support segmentation masks.
- Display defect type and confidence.
- Allow Confirm.
- Allow Reject.
- Allow Modify.
- Preserve original prediction.
- Capture review comments.
- Display model version for audit users where appropriate.

The UI must clearly distinguish:

AI Suggested Defect

from:

Confirmed Inspection Defect

Implement responsive and maintainable components.

Phase 8 — Human Feedback and Training Candidate Loop

Implement continuous learning candidate generation.

Automatically consider images when:

- AI prediction is rejected.
- AI prediction is modified.
- Confidence is below a configurable threshold.
- A manual defect is added without an AI prediction.
- Random sampling is configured.

Create an AITrainingCandidate workflow.

Do not automatically add production images directly to the training dataset.

All images must pass annotation and review.

Phase 9 — Model Registry and Governance

Implement:

- Model registration
- Model versions
- Dataset linkage

- Training run linkage
- Metrics
- Approval workflow
- Staging deployment
- Production deployment
- Rollback

Only one approved model should be active for a defined model scope unless explicitly supporting A/B testing.

Every prediction must reference the exact model version used.

Phase 10 — Monitoring, Security, and Production Readiness

Add:

- Structured logging
- Health checks
- Metrics
- AI job monitoring
- Inference latency monitoring
- Failure alerts
- Authentication
- Authorization
- Audit logging
- Secure image access
- Configuration validation
- Docker deployment

Provide:

- Docker Compose for local development
- Environment variable documentation
- Health check endpoints
- Production deployment checklist
- Database backup considerations
- Model rollback procedure

Phase 11 — Testing and Quality Gates

Implement:

- Unit tests
- Integration tests
- API tests
- AI pipeline validation tests
- Model metadata validation tests
- End-to-end workflow tests

Define quality gates for:

- Dataset validation
- Model evaluation
- Model approval
- Production deployment

Expected Response Format for Every Phase

For each implementation phase, respond using exactly this structure:

1. Phase Objective
2. Architecture Decision
3. Prerequisites
4. Database Changes
5. Backend Implementation
6. API Contracts
7. Frontend Changes
8. Background Processing
9. Tests
10. Local Run Instructions
11. Validation Checklist
12. Production Considerations
13. Next Phase

Start with Phase 1 only.

Do not proceed to the next phase until Phase 1 is complete and reviewed.

22. Final Recommendation

Build the platform incrementally.

The first complete vertical slice should be:

```

Upload Wall Image
  ↓
Store Image
  ↓
Annotate Scratch Examples
  ↓
Train YOLO
  ↓
Export ONNX
  ↓
Run Inference
  ↓
Display AI Prediction
  ↓
Inspector Confirms / Rejects
  ↓
Save Audit Trail
  
```

After this works reliably, expand into:

```

Multiple Defect Classes
  ↓
Segmentation
  ↓
Training Candidate Queue
  ↓
Label Studio Review
  ↓
Dataset Versioning
  ↓
Retraining
  ↓
Model Governance
  ↓
Production Scaling
  
```

The recommended overall stack is:

Label Studio → Python ML Pipeline → YOLO → ONNX → ASP.NET Core Background Processing and Integration → React Human Review → SQL Server Audit and Governance

This approach uses established software engineering principles while keeping the ML lifecycle separate, reproducible, traceable, and production-ready.

Today 5:37 PM



Can I make this product as saas to sale out in the market

Yes — **you can absolutely turn this into a SaaS product and sell it commercially.**

In fact, I think the architecture we designed is stronger if you intentionally evolve it from an **internal property inspection system** into a **multi-tenant AI Property Inspection SaaS platform**.

The main change is this:

Don't build it as **your company's inspection application with AI added**.

Build the AI and inspection capabilities as a **tenant-aware platform** that different property

companies can configure and use independently.

1. The SaaS product vision

Your product could be positioned as:

AI-Powered Property Inspection & Defect Management Platform

Potential customers:

- Property developers
- Property management companies
- Facility management companies
- Real estate companies
- Property handover companies
- Construction companies
- Maintenance companies
- Hotels and hospitality groups
- Insurance/property assessment companies

The core value proposition could be:

Inspect properties faster, detect defects with AI, maintain a complete audit trail, and continuously improve defect detection using your organization's inspection data.

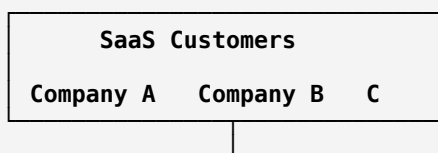
2. Your SaaS architecture

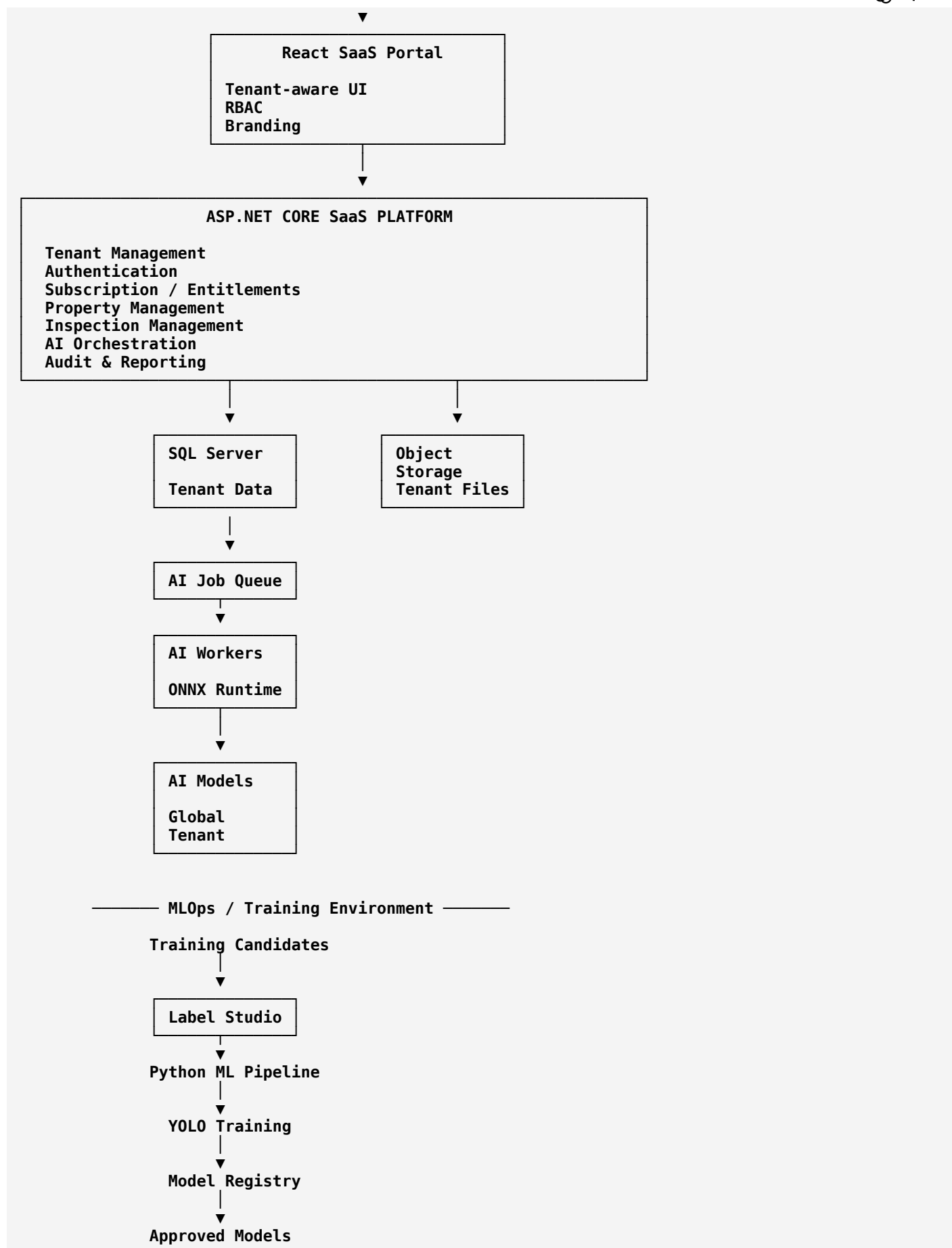
Your current architecture:

```

Label Studio
  ↓
YOLO
  ↓
ONNX
  ↓
ASP.NET Core
  ↓
React
  ↓
SQL Server
  
```

Should evolve into:





3. The most important SaaS decision: Multi-tenancy

You need to introduce a top-level entity:

Tenant

Everything important belongs to a tenant.

Tenant

- Users
- Properties
- Projects
- Inspections
- Inspection Images
- Defects
- AI Predictions
- Reports
- Subscription

For example:

Tenant A – ABC Properties

- Dubai Marina Tower
- Palm Residence
- Business Bay Offices

Tenant B – XYZ Facilities

- Hotel One
- Hotel Two
- Shopping Mall

Tenant C – Developer Company

- Project Alpha
- Project Beta

Tenant A must never be able to access Tenant B's data.

This must be enforced in:

- Database queries
- APIs
- File storage
- AI jobs
- Reports
- Background workers
- Caches
- Search indexes

Not only in the React UI.

4. Recommended database approach

For your first SaaS version, I recommend:

Shared SQL Server database with TenantID

```
Tenant
├── TenantID
├── Name
├── Status
└── SubscriptionPlan
```

```
Property
├── PropertyID
├── TenantID
└── Name
```

```
Inspection
├── InspectionID
├── TenantID
└── PropertyID
```

```
InspectionImage
├── InspectionImageID
├── TenantID
└── StorageKey
```

```
AIPrediction
├── AIPredictionID
├── TenantID
└── InspectionImageID
```

Every tenant-owned table should have:

```
SQL
TenantID
```

And your repository/application layer should automatically apply tenant filtering.

Conceptually:

```
Current User
┆
┆   ↓
┆ Tenant Context
┆   ↓
┆ Tenant ID = 101
┆   ↓
┆ Every Query Automatically Scoped
┆   ↓
┆ WHERE TenantID = 101
```

Do **not** depend on developers remembering to add **TenantID** manually to every query.

5. Your AI models can become a major product differentiator

I recommend supporting three model levels.

Level 1 — Global model

You train one general model:

PropertyDefect-Global-v1

It detects:

- Scratch
- Crack
- Stain
- Paint damage
- Hole
- Dampness

All standard SaaS customers can use it.

All Tenants
↓
Global AI Model

This is the best starting point.

Level 2 — Tenant-specific configuration

Different companies may use different terminology.

For example:

Customer A

Minor Scratch
Major Scratch
Paint Damage

Customer B

Cosmetic Damage
Structural Damage

Your AI may still detect the same underlying defect:

AI → Scratch

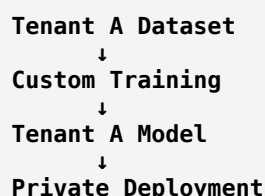
Then your SaaS configuration maps it:

Tenant A → Minor Scratch
Tenant B → Cosmetic Damage

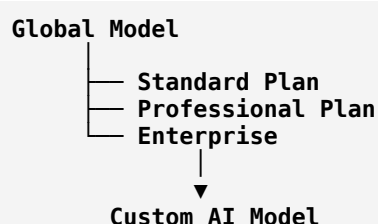
This allows customization **without retraining the model**.

Level 3 — Tenant-specific model

Premium customers could have:



This can become an Enterprise feature.



That is potentially a strong premium offering.

6. Very important: customer data and AI training

Do **not** automatically use all customer inspection images to train your global model.

Instead, make this explicit.

```

Tenant AI Settings

[ ] Allow my anonymized inspection data
    to improve the global AI model
  
```

Or:

- Do not use our data for training
- Allow anonymized data for global model improvement
- Use our data only for our private AI model

This becomes important commercially and contractually.

7. SaaS subscription model

You could create plans like:

Feature	Starter	Professional	Enterprise
Users	Limited	More	Custom

Feature	Starter	Professional	Enterprise
Properties	Limited	High	Unlimited
Inspections	Monthly limit	Higher	Custom
Image storage	Limited	Higher	Custom
AI analysis	Limited	Included quota	Custom
AI defect detection	Yes	Yes	Yes
Custom defect templates	Limited	Yes	Yes
Custom workflows	No	Yes	Yes
API access	No	Yes	Yes
Custom AI model	No	No	Yes
Private deployment	No	No	Yes
Dedicated environment	No	No	Optional

I would make **AI analysis usage-based** internally.

For example:

Subscription
+
AI Analysis Credits

Each image analyzed consumes a unit.

1 Image
↓
AI Analysis
↓
1 Credit

This protects you from a customer uploading millions of images on a low-cost plan.

8. Separate your product into modules

Don't sell only "YOLO property inspection."

Sell a complete business platform.

Module 1 — Property & Asset Management

Properties
Buildings
Units
Areas
Assets

Module 2 — Inspection Management

Templates
 Checklists
 Activities
 Inspection Items
 Schedules
 Assignments

Module 3 — Defect Management

Defects
 Severity
 Priority
 Responsibility
 Status
 Photos
 Comments

Module 4 — AI Inspection

AI Detection
 AI Suggestions
 Confidence
 Inspector Review
 Feedback

Module 5 — Handover Management

Snag Lists
 Handover
 Acceptance
 Re-inspection
 Closure

Module 6 — Analytics

Defect Trends
 AI Accuracy
 Inspector Productivity
 Property Quality
 Recurring Defects

This is much harder for competitors to replace than just an API that says:

Upload Image → Scratch Detected

9. Commercial licensing: a critical issue

Before selling a proprietary SaaS, you need to review the licenses of your ML components.

The biggest issue in the proposed stack is Ultralytics YOLO licensing. Ultralytics currently states that proprietary commercial products, SaaS platforms, APIs, and cloud systems using its YOLO

software/models require an Enterprise License unless the entire relevant project is released under AGPL-3.0. This is something you should resolve before commercial launch. [Ultralytics+1](#)

By contrast, **ONNX Runtime is MIT licensed**, and its repository explicitly permits commercial use, modification, distribution, and sale subject to the license terms. [GitHub+1](#)

So your commercial architecture should include a licensing decision:

```
Option A
Ultralytics Enterprise License
↓
Proprietary SaaS

Option B
Use a commercially suitable alternative
model/framework after legal review

Option C
Open-source the relevant product under AGPL
```

For a proprietary B2B SaaS, **Option A or a legally reviewed alternative is generally the practical path.**

This is a business/legal decision worth making early, before investing heavily in proprietary training and deployment infrastructure.

10. Label Studio in a SaaS architecture

I would **not expose Label Studio directly to normal SaaS customers initially.**

Keep it as your internal ML operation:

```
Customer
↓
Property Inspection SaaS
↓
Training Candidate Queue
↓
Internal ML Team
↓
Label Studio
```

Label Studio Community Edition documentation notes that users have the same functionality and can see all projects, while role-based access is an Enterprise capability. That makes tenant isolation and workflow design especially important if you ever expose annotation directly to multiple customers.

 [Label Studio](#)

Later, an Enterprise customer could receive:

```
Dedicated Label Studio Environment
```

for private model training.

11. Recommended SaaS product architecture

I would organize your product like this:

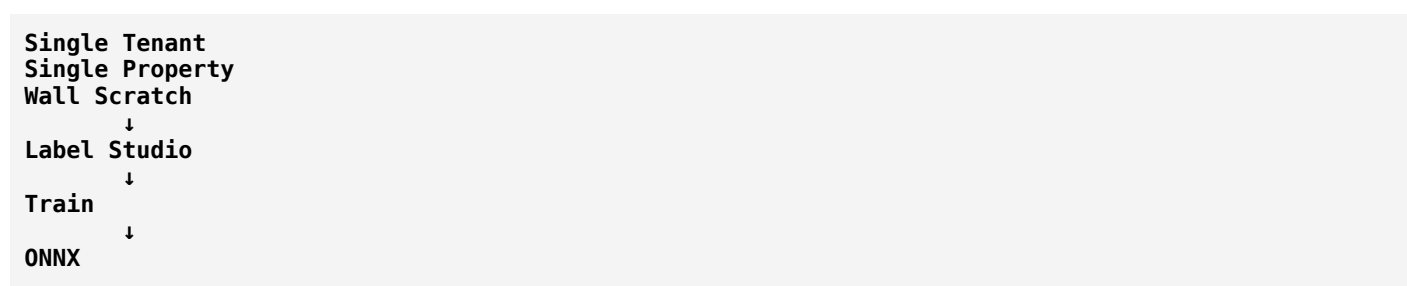


12. Recommended implementation strategy

I would **not start by building multi-tenancy, billing, custom AI models, and 20 defect types simultaneously.**

Build in this order.

Stage 1 — Prove the AI



↓
Inspector Review

Stage 2 — Build the core platform

Properties
Inspections
Images
Defects
AI Predictions
Audit

Stage 3 — Make it SaaS-ready

Tenant
Users
RBAC
Tenant Isolation

Stage 4 — Add commercial controls

Plans
Subscriptions
Usage
AI Credits
Feature Flags

Stage 5 — Enterprise capabilities

SSO
Custom Branding
API Access
Custom Workflows
Custom AI Models
Private Deployment

My honest assessment

Yes, this has real SaaS potential, particularly because you have a domain advantage in property inspection and handover workflows.

The opportunity is **not simply YOLO defect detection**. Many teams can connect an image model to an API.

Your defensible product is the combination of:

**Property inspection workflow + domain-specific defect intelligence + human verification
+ audit trail + proprietary annotated data + continuous model improvement.**

Your long-term moat can become the dataset and feedback loop:

More Customers
↓
More Inspections
↓
More Reviewed Images
↓
Better Ground Truth

↓
Better Models
↓
Better Detection
↓
More Product Value

That is the direction I would recommend. The **next architectural step** should be to redesign the blueprint we created into a **multi-tenant SaaS architecture**, including tenant isolation, subscription/usage metering, model ownership, and a phased MVP-to-enterprise roadmap.

ChatGPT can make mistakes. Check important info.